

Airline Reservation Management System: A Secure Spring Boot Web Application

Project Team

Department of Computer Engineering

Academic Project Report, 2025-2026

Abstract- This paper presents the Airline Reservation Management System (ARMS), a web-based platform for flight discovery, passenger registration, fare estimation, reservation processing, and administrative monitoring. The application is implemented using Spring Boot 3, Thymeleaf, Spring Security, JPA, MySQL, and client-side JavaScript. ARMS maintains a clear separation between presentation, business, and persistence layers while supporting route-based flight search, distance-driven fare calculation, seat availability checks, booking confirmation generation, and dashboard-level operational summaries. The system uses JWT-backed authentication for account security and persists key entities such as users, flights, and bookings in a relational database. The resulting platform demonstrates how a modern Java web stack can be used to digitalize airline booking workflows in an accessible and modular manner.

Index Terms- Airline Reservation, Spring Boot, JWT Authentication, Thymeleaf, MySQL, Booking System, Admin Dashboard.

I. INTRODUCTION

Air transportation systems demand timely reservation handling, consistent seat inventory management, and secure access control for both passengers and administrators. Traditional manual or semi-digital reservation workflows often suffer from delayed updates, duplicate work, and limited visibility into real-time seat status. The Airline Reservation Management System addresses these limitations through a centralized web platform that allows passengers to register, browse routes, verify prices, and complete bookings while enabling administrators to monitor flights, passengers, revenue, and booking trends from a unified interface.

A. Problem Statement

A flight booking platform must validate origin and destination pairs, protect user accounts, compute fares consistently, avoid overbooking, and provide immediate booking feedback. In many academic prototypes, these concerns are handled in isolation, resulting in fragmented flows and weak traceability across the reservation lifecycle. The core problem addressed by ARMS is the integration of secure authentication, dynamic route selection, accurate seat tracking, and operational visibility in one maintainable application.

B. Motivation

The project is motivated by the need to model a realistic service workflow using enterprise Java technologies. Airline booking is a familiar but technically rich domain that combines data validation, transactional updates, user management, and dashboard-style reporting. Building ARMS provides hands-on experience with full-stack development while also demonstrating how core software engineering principles such as modularization, persistence, and role-aware access can be applied to a practical transport use case.

C. Project Scope and Objectives

The scope of ARMS includes passenger registration, secure login, route browsing, seat-aware booking, fare estimation, booking

confirmation, and administrative monitoring. The primary objectives are to: (i) provide a smooth booking experience for end users; (ii) maintain route and seat information in a relational database; (iii) secure identity flows through JWT-based authentication; (iv) support administrator functions for adding flights and viewing demand patterns; and (v) keep the codebase extensible for future features such as cancellation, payment gateways, and notification services.

II. LITERATURE REVIEW

Reservation systems are a standard case study in web engineering because they combine multi-user access, transactional integrity, and data-centric workflows. Prior studies on transportation booking platforms emphasize the importance of normalized relational data models, search usability, and security-first session handling. Within the Java ecosystem, Spring Boot is widely adopted for rapid backend development, while Thymeleaf remains a practical templating choice for server-rendered pages. JPA-based persistence and MySQL-backed storage are also common patterns for academic reservation systems due to their approachable development model and strong community support.

The ARMS implementation aligns with these patterns by using entity-driven persistence for core business objects, controller-service layering for request handling, and a template-based interface for passengers and administrators. Unlike a purely static prototype, the project goes further by preserving live seat counts, assigning booking references, and exposing dashboard metrics such as total bookings, revenue, and route popularity.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

ARMS follows a layered architecture that separates user interaction, business logic, and persistence concerns. This structure improves maintainability and makes it easier to evolve individual modules without destabilizing the overall reservation flow.

A. Architectural Framework

The presentation layer consists of Thymeleaf templates under passenger and administrator views, styled with CSS and supported by JavaScript for client-side interactions. The application layer includes controllers for routing requests and services for implementing business rules such as price estimation, seat validation, and booking persistence. The data layer uses JPA repositories over MySQL for storing flights, users, and bookings. Security concerns are managed by Spring Security and JWT filters, allowing the system to attach identity information to protected operations.

B. Execution Methodology

The workflow begins when a passenger registers or authenticates into the system. After login, the user selects an origin, destination, number of seats, and travel date. The booking service first rejects invalid requests, such as identical source and destination values or dates earlier than the current day. It then queries the flight repository for a matching route, calculates the fare using the configured base fare per kilometer, and checks whether sufficient seats remain. If validation succeeds, the booking record is stored, the flight inventory is reduced, and a confirmation response containing the booking reference and ticket number is generated.

C. User Interface Design

The user-facing interface is organized into pages for home, route selection, payment, ticket display, booking confirmation, and booking history. On the administrative side, the dashboard summarizes revenue, booking counts, passenger totals, low-seat

flights, and popular routes. This division enables each actor to access only the information relevant to their role while keeping the navigation model straightforward for an academic demonstration system.

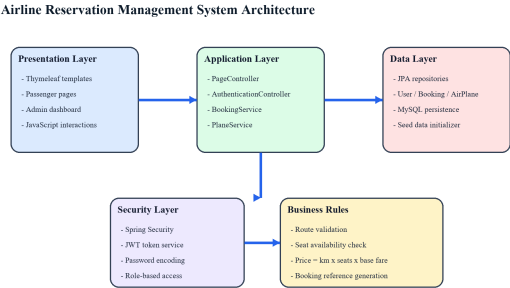


Fig. 1. Layered view of the ARMS web application and its key responsibilities.

Fig. 1. Layered architecture of the Airline Reservation Management System.

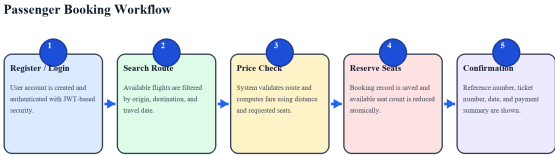


Fig. 2. Sequential workflow followed by a passenger from authentication to first booking confirmation.

Fig. 2. Booking workflow executed by ARMS during reservation processing.

IV. IMPLEMENTATION DETAILS

A. Technological Stack and Environment

The backend is implemented in Java 17 using Spring Boot 3.0.1 with dependencies for Spring Web, Spring Data JPA, Spring Security, Thymeleaf, Mail, JWT processing, and Lombok. MySQL serves as the persistent data store. Frontend pages are built with HTML templates, CSS, Bootstrap-based styling assets, and JavaScript. This combination provides a balanced stack for rapid development while preserving strong support for structured application design.

B. Data Structures and Object Modeling

Three main entities define the data model. The User entity stores identity fields, encrypted passwords, and role information for both passengers and administrators. The AirPlane entity stores route endpoints, available seat count, and flight distance in kilometers. The Booking entity records the selected route, passenger email, seat count, travel date, total price, booking reference, and ticket number. These classes are mapped to relational tables and accessed through dedicated repository interfaces.

C. Security and Authentication

Authentication is implemented through Spring Security with a JWT filter placed before the username-password authentication filter. During registration, passwords are encoded before persistence and each user is assigned either a passenger or administrator role. During login, credentials are authenticated through the configured authentication manager, and the JWT service generates a token for subsequent requests. This design supports stateless security handling and keeps protected business operations tied to the active user identity.

D. Core Booking Logic

The BookingService acts as the primary business module for reservation processing. It calculates price using a base fare of 12.0 per kilometer multiplied by the requested number of seats, validates that the travel date is not in the past, checks that source

and destination are different, and verifies that a matching flight exists. When a booking is confirmed, the service creates a unique booking reference and ticket number, stores the transaction, and decrements the corresponding flight's available seat count. This logic prevents overbooking while maintaining a traceable reservation record.

E. Administrative Features

The administrative module exposes dashboard-level summaries derived from repository data. It reports total bookings, total revenue, total passengers, total flights, flights with low remaining seat counts, and a ranked list of heavily booked routes. Administrators may also add new flights through a dedicated form that captures route endpoints, seat capacity, and route distance. A demo data initializer seeds representative flight pairs and baseline user accounts to simplify testing and presentation.

V. RESULTS AND DISCUSSION

Functional review of the application shows that the system successfully supports the main reservation lifecycle from user onboarding to booking confirmation. Route lists are populated from stored flight records, seat inventory updates after booking, and generated identifiers improve traceability of each transaction. The administrator views further convert raw repository data into immediately useful indicators for operational monitoring.

A. Functional Outcomes

The completed application demonstrates that a server-rendered Spring Boot solution can deliver a coherent airline reservation experience without requiring a separate single-page frontend. Users are able to discover routes, estimate price, reserve seats, and revisit booking data, while administrators can inspect route pressure and revenue summaries. The modular code organization also supports incremental addition of new service logic.

Module	Primary Responsibility	Key Technologies	Observed Benefit
Authentication	Register users and generate tokens	Spring Security, JWT	Secures access
Flight Management	Store route and seat data	JPA, MySQL	Centralized inventory
Booking Engine	Validate and save reservations	Spring Services	Seat-aware booking
Admin Dashboard	Summarize routes and revenue	Thymeleaf, repositories	Operational visibility

VI. CONCLUSION AND FUTURE WORK

The Airline Reservation Management System demonstrates a practical application of secure full-stack Java development to a real-world reservation domain. By combining Spring Boot, MySQL, Thymeleaf, and JWT-backed security, the project achieves a functional workflow for registration, route discovery, fare estimation, seat-aware booking, and administrative monitoring. The resulting system is suitable as both an academic project and a foundation for further product-level refinement.

Future improvements may include online payment gateway integration, email notifications, ticket cancellation with seat restoration, route schedules with departure times, stronger transactional safeguards around concurrent booking, and analytics dashboards with chart-based visualizations.

REFERENCES

- [1] C. Richardson, Microservices Patterns. Manning Publications, 2018.
- [2] C. Walls, Spring Boot in Action. Manning Publications, 2016.
- [3] R. Johnson et al., Professional Java Development with the Spring Framework. Wrox, 2005.
- [4] Oracle, 'The Java Language Specification, Java SE 17 Edition,' 2021.

[5] P. DuBois, MySQL. Addison-Wesley, 2021.